

Gabriela Mora, V-31.541.893
Lucía Martínez, V-31.906.577
Maivet Pereira, V-30.814.708

Simulador de Caché de Correspondencia Directa y Asociativa por Vías

Arquitectura del Computador

1 de Septiembre de 2026

El Problema

Descripción del problema

La memoria caché es un pequeño espacio de almacenamiento ultrarrápido que el procesador utiliza para evitar ir constantemente a la memoria principal, que es mucho más lenta. El problema es que su tamaño es limitado y, si no se gestiona bien, el procesador pasa mucho tiempo esperando datos. Este proyecto simula su comportamiento para entender cómo optimizar su uso.

Objetivos del proyecto

Desarrollar un simulador en C++ que modele el funcionamiento de una caché, permitiendo probar diferentes configuraciones (mapeo directo y asociativo por vías) y analizar su rendimiento en términos de aciertos y fallos.

Alcance

Nuestro simulador trabaja con dos tipos de organización: la caché de mapeo directo, donde cada bloque de memoria solo puede ir a un lugar específico, y la

caché asociativa por vías, que ofrece más flexibilidad al permitir que un bloque se ubique en varias líneas dentro de un conjunto.

Técnicas Computacionales para su Solución

Estructuras de Datos en C++

La caché se modela utilizando una matriz de estructuras `lineaCache` con `std::vector`:

- `struct lineaCache`: Representa un bloque de caché, conteniendo los campos `datos`, `tag`, `ultimoUso` y `valido`.

```
struct lineaCache
{
    int datos;
    int tag;
    int ultimoUso; // contador de reloj para LRU
    bool valido;

    lineaCache(): datos(0), tag(-1), ultimoUso(0), valido(false){}
};
```

- `vector<vector<lineaCache>> conjuntos`;: Se utiliza una matriz de vector donde cada fila (conjunto) contiene un vector de líneas (vías). Esto permite un acceso directo por índice en tiempo constante $O(1)$.
- `vector<int> direcciones`;: Un arreglo dinámico para almacenar las direcciones ingresadas por el usuario.

Algoritmos de Simulación

Cálculo de Índice y Tag: Se utilizan las operaciones aritméticas (`%` y `/`) para calcular el conjunto y la etiqueta a partir de la dirección.

LRU (Least Recently Used)

El LRU es el algoritmo de reemplazo que usamos para decidir qué línea eliminar cuando la caché está llena. Su funcionamiento es sencillo: cada línea de caché tiene un contador llamado `ultimoUso` que se actualiza cada vez que se accede a ella. Cuando ocurre un fallo y no hay líneas libres, simplemente reemplazamos la línea con el valor más bajo en `ultimoUso`, es decir, la que lleva más tiempo sin ser utilizada.

```

// si no hay lineas vacias, se aplica la politica de reemplazo LRU
int lru = 0;

// recorre todas las vias para encontrar la de menor ultimoUso
for (int i = 1; i < vias; i++)
{
    if (conjuntos[indice][i].ultimoUso < conjuntos[indice][lru].ultimoUso)
    {
        lru = i;
    }
}

// reemplaza la linea LRU con el nuevo bloque
conjuntos[indice][lru].tag = tag;
conjuntos[indice][lru].datos = direccion;
conjuntos[indice][lru].valido = true;
conjuntos[indice][lru].ultimoUso = reloj;

```

Prefetching (Anticipo de Datos)

El prefetching es una técnica que usamos para anticipar las necesidades del procesador. Cuando se produce un fallo y el prefetch está activado, no solo cargamos la dirección solicitada, sino también las direcciones siguientes (direccion + 1, +2, etc.). Esto aprovecha la localidad espacial, ya que los programas suelen acceder a datos cercanos en memoria. El resultado es una reducción significativa de los fallos, especialmente en patrones de acceso secuencial.

```

void precargar(int direccion, int cantidad)
{
    for (int p = 1; p <= cantidad; p++)
    {
        int dir = direccion + p;
        int indice = dir % numConjuntos;
        int tag = dir / numConjuntos;

        bool encontrado = false;

        for (int i = 0; i < vias; i++)
        {
            if (conjuntos[indice][i].valido && conjuntos[indice][i].tag == tag)
            {
                encontrado = true;
                conjuntos[indice][i].ultimoUso = reloj;
            }
        }

        if (!encontrado)
        {
            for (int i = 0; i < vias; i++)
            {
                if (!conjuntos[indice][i].valido)
                {
                    conjuntos[indice][i].tag = tag;

```

```
conjuntos[indice][i].datos = dir;
conjuntos[indice][i].valido = true;
conjuntos[indice][i].ultimoUso = reloj;
break;
    }
}
}
```

Organización de la Caché y Experimentación

El simulador mantiene un total de 16 líneas, permitiendo probar diferentes organizaciones:

- **Mapecto Directo:** Cuando se usa $vias = 1$, cada direccin solo puede residir en un conjunto especfico.
- **Asociativa por Vias:** Al aumentar el nmero de vas (2, 4, 8, 16), los conjuntos se reducen, pero hay ms flexibilidad para ubicar los datos, lo que reduce los fallos de conflicto.

Interfaz de Usuario

Se implementó un menú interactivo que permite:

- Ingresar direcciones manualmente.
- Ver las estadísticas de la caché (aciertos, fallos, tasa).
- Comparar el rendimiento de diferentes esquemas (mapeo directo vs. asociativa) con o sin prefetch.

Análisis y Discusión de Resultados

Experimentos Realizados

Probamos tres tipos de secuencias: consecutiva (buena localidad), repetitiva (localidad mixta) y aleatoria (sin localidad). Cada una se probó con 1, 2, 4 y 8 vías, con y sin prefetch.

Análisis

La caché de 8 vías dio la mejor tasa en todos los casos por su mayor flexibilidad para ubicar datos. El prefetch mejoró los resultados en secuencias consecutiva y

repetitiva (aprovechando la localidad espacial), pero no tuvo efecto en la aleatoria. Esto confirma que el prefetch es útil solo cuando los accesos son predecibles.

Table 1: Tasas de acierto sin prefetch

Secuencia	Directa	2 vías	4 vías	8 vías
Consecutiva	30%	50%	60%	70%
Repetitiva	25%	50%	62%	75%
Aleatoria	20%	30%	30%	40%

Table 2: Tasas de acierto con prefetch (1 dirección)

Secuencia	Directa	2 vías	4 vías	8 vías
Consecutiva	60%	80%	90%	100%
Repetitiva	50%	75%	87%	100%
Aleatoria	20%	30%	30%	40%

Conclusiones

Este proyecto nos permitió comprender cómo influye la organización de la caché en el rendimiento del sistema. Comprobamos que la asociatividad por vías reduce los fallos de conflicto y que el prefetch mejora la tasa de aciertos cuando los accesos son secuenciales, pero no aporta beneficio en patrones aleatorios.

Los objetivos se cumplieron: logramos un simulador funcional que permite probar configuraciones, calcular estadísticas y comparar esquemas con y sin prefetch.

Las principales dificultades fueron implementar LRU y coordinar el trabajo en equipo, pero nos ayudaron a mejorar nuestras habilidades de programación y colaboración.